

Informe Proyecto Final

Carlos Alberto Zapata Rangel, Johan Samuel Velásquez

Programacion I

Grupo 01D

1. Pensamiento Computacional Aplicado

El desarrollo del proyecto requirió aplicar los cuatro pilares del pensamiento computacional para resolver el problema de gestionar un sistema de monederos digitales con clientes, transacciones y una interfaz gráfica.

1.1 Abstracción

La abstracción consistió en identificar los elementos esenciales del sistema y omitir detalles irrelevantes para modelar un monedero digital.

Se abstrajeron los siguientes conceptos principales:

Cliente: entidad que posee datos personales y una lista de monederos.

Monedero: representa una cuenta digital con saldo y un historial de transacciones.

Transacción: acción realizada sobre un monedero (depósito, retiro, transferencia).

SistemaMonedero: clase que gestiona clientes, transacciones y reglas de negocio.

Notificador: abstracción de un servicio externo para enviar alertas.

Interfaz gráfica (JavaFX): capa que interactúa con el usuario final.

Cada clase se creó concentrándose en su propósito fundamental.

Por ejemplo, la clase Transaccion solo guarda la información común (monto, fecha, monederos involucrados), delegando su comportamiento específico a las subclases.

1.2 Descomposición

El sistema completo se descompuso en módulos más pequeños y manejables, siguiendo el principio de responsabilidad única:

Módulo de Clientes: registro y búsqueda.

Módulo de Monederos: saldo, historial y operaciones.

Módulo de Transacciones: depósito, retiro, transferencia.

Módulo de Puntos y Rangos: reglas de beneficios.

Módulo de Transacciones Programadas: almacenamiento y ejecución.

Módulo de Notificación: alertas por saldo bajo.

Capa Controlador: interacción entre interfaz y lógica.

Capa de Vista (FXML): presentación visual.

Gracias a la descomposición fue posible corregir errores por módulos, como fallos en actualización de saldo, sin afectar el resto de funcionalidades.

1.3 Reconocimiento de Patrones

Durante el diseño se identificaron patrones comunes que facilitaron la implementación:

Las clases:

- Deposito
- Retiro
- Transferencia

comparten una estructura similar (ejecutar la transacción), pero cada una implementa pasos específicos.

Se reconoció este patrón y se manejó mediante herencia y métodos abstractos.

Se separaron:

Modelo: lógica del negocio (clases cliente, monedero, transacción)

Controlador: interacción con los FXML

Vista: archivos. fxml estilizados con CSS

2. Pruebas Realizadas

Se implementaron pruebas unitarias para validar la lógica del sistema.

A continuación, se describen las pruebas más importantes:

2.1 Prueba — Verificación Recursiva de Transacciones

Se evaluó el método `verificarLista()` de `VerificadorTransacciones` que usa recursividad para validar montos positivos.

```
@Test
void verificarLista_devuelveTrue_siTodosLosMontosSonPositivos() {
    List<Transaccion> lista = new ArrayList<>();
    lista.add(crearTransaccionValida(50));
    lista.add(crearTransaccionValida(20));
    lista.add(crearTransaccionValida(10));

    boolean resultado = VerificadorTransacciones.verificarLista(lista);

    assertTrue(resultado);
}

@Test
void verificarLista_devuelveFalse_siAlgunaTransaccionTieneMontoCeroONegativo() {
    List<Transaccion> lista = new ArrayList<>();
    lista.add(crearTransaccionValida(30));
    lista.add(crearTransaccionInvalida(0)); // aquí está el monto inválido
    lista.add(crearTransaccionValida(10));

    boolean resultado = VerificadorTransacciones.verificarLista(lista);

    assertFalse(resultado);
}
```

Resultado:

- El método devuelve true si todos los montos son positivos.
- Devuelve false si existe alguna transacción con monto menor o igual a cero.

2.2 Prueba — Registrar Cliente en el Sistema

Se probó el método `registrarCliente()`:

```
@Test no usages johansamuel
void registrarCliente_agregaClienteAlaLista() {
    Notificador notificador = crearNotificadorMock();
    SistemaMonedero sistema = new SistemaMonedero(notificador);

    Cliente c = new Cliente("1", "Carlos", "1234");

    sistema.registrarCliente(c);

    List<Cliente> clientes = sistema.getClientes();

    assertEquals(1, clientes.size());
    assertEquals(c, clientes.get(0));
}
```

Resultado:

- El cliente es agregado correctamente a la lista interna.
- El método `getClientes()` devuelve una lista con el cliente agregado.

2.3 Prueba — Búsqueda de Cliente por Nombre

Se probó el método `buscarClientePorNombre()`:

```
@test no usages johansamuel
void buscarClientePorNombre_encuentraClienteCorrecto() {
    Notificador notificador = crearNotificadorMock();
    SistemaMonedero sistema = new SistemaMonedero(notificador);

    Cliente c1 = new Cliente("1", "Carlos", "1234");
    Cliente c2 = new Cliente("2", "Samuel", "abcd");

    sistema.registrarCliente(c1);
    sistema.registrarCliente(c2);

    Cliente resultado = sistema.buscarClientePorNombre("Samuel");

    assertNotNull(resultado);
    assertEquals("2", resultado.getId());
    assertEquals("Samuel", resultado.getNombre());
}
```

Resultado:

- Si el nombre coincide, devuelve el cliente correcto.
- Si no existe, devuelve null.

3. Análisis Técnico de Programación Orientada a Objetos

En el proyecto se aplicaron los cuatro principios fundamentales de POO.

3.1 Abstracción

Consiste en modelar las entidades con sus atributos y comportamientos esenciales.

Ejemplos en el proyecto:

Clase Transaccion:

Atributos universales: monto, fecha, monedero origen y destino.

Método abstracto `getDescripcion()` que obliga a describir cada operación.

Clase Monedero:

Solo expone métodos acreditar y debitar, ocultando detalles internos.

Cliente:

Guarda información personal y reglas de puntos.

Gracias a la abstracción, el sistema es fácil de extender (por ejemplo, agregar un nuevo tipo de transacción).

3.2 Encapsulamiento

Se protegieron los datos del sistema usando:

- Atributos privados en todas las clases.
- Métodos públicos para modificarlos de forma controlada.
- Validación interna al debitar dinero.
- Historial devuelto como lista inmodificable con `Collections.unmodifiableList()`.

3.3 Herencia

Se utilizó para evitar duplicación de código y organizar comportamientos comunes.

Jerarquías:

1. Transaccion (abstracta) → Deposito, Retiro, Transferencia

- Cada una implementa `getDescripcion()` según su tipo.
- Comparten atributos y lógica base.

2. Monedero (abstracta) → MonederoAhorro, MonederoGasto

- Permite definir comportamientos distintos por tipo.

3. Notificador → NotificadorEmail

- Facilita cambiar el método de notificación sin modificar el sistema.

3.4 Polimorfismo

Se utilizó polimorfismo tanto de sobreescritura como de sustitución:

- **Polimorfismo por Sobreescritura**

Cada transacción implementa su propia descripción:

@Override

```
public String getDescripcion() {  
    return "Depósito de $" + monto;  
}
```

- **Polimorfismo por Sustitución**

El sistema trata todas las transacciones como tipo base Transacción, sin importar cuál sea:

```
Transaccion t = new Deposito(100, monedero);  
t.ejecutar();
```

El método ejecutado dependerá del tipo real del objeto.

- **Polimorfismo en Notificador**

```
sistema.setNotificador(new NotificadorEmail());
```

Nuestro sistema no conoce el tipo exacto de notificador; solo sabe que implementa el método notificar().

Conclusión

Nuestro proyecto del **Monedero Virtual** implementa correctamente los principios de pensamiento computacional y los pilares de programación orientada a objetos.

La descomposición modular nos permitió construir un sistema:

- **Extensible**
- **Mantenible**
- **Seguro**
- **Fácil de expandir**

Además, la interfaz JavaFX integra correctamente la lógica del negocio con una representación visual clara para el usuario.

